

# Mean Shift Clustering on Circular–Linear Data via Cartesian Embedding

Yang Tang

November 23, 2025

## 1 Introduction

This report presents a clustering study on circular–linear data using the Mean Shift algorithm. Because Mean Shift operates in Euclidean space, the original polar measurements  $(\theta, r)$  are first converted into 2D Cartesian coordinates. Clustering is then performed using both Epanechnikov and Gaussian kernels, and the resulting clusters are evaluated for different bandwidths.

## 2 Background

### 2.1 Kernel

In the Mean Shift procedure, a kernel function defines the influence of neighboring points on the density estimation. Two commonly used kernels are:

1. Epanechnikov kernel:

$$K(x) = \begin{cases} 1 - x, & \text{if } x \in [0, 1), \\ 0, & \text{if } x \geq 1. \end{cases} \quad (1)$$

2. Gaussian Kernel:

$$K(x) = e^{-x/2}. \quad (2)$$

The Gaussian kernel provides smooth decay, while the Epanechnikov kernel is computationally efficient. They are sufficient for most applications[5].

### 2.2 Mean Shift

Mean Shift is a non-parametric, iterative clustering method that does not require specifying the number of clusters in advance. Mean Shift was first introduced by Fukunaga and

Hostetler[7] with a goal of estimating the gradient of a multivariate density, later adapted by Cheng [3] for the purpose of image segmentation.

The main idea behind Mean Shift is to interpret the points as samples from an unknown density function, whose modes correspond to cluster centers. For each point, one performs a gradient ascent procedure on the estimated density until convergence. The math definition and convergence proof can be found in [4] and [1]

Given a dataset  $X$  consists of  $n$  points in  $R^d$ , the mean shift vector  $\mathbf{m}(\mathbf{x})$  at point  $\mathbf{x}$  is:

$$\mathbf{m}(\mathbf{x}) = \frac{\sum_{i=1}^n \mathbf{x}_i K' \left( \left\| \frac{\mathbf{x} - \mathbf{x}_i}{h} \right\|^2 \right)}{\sum_{i=1}^n K' \left( \left\| \frac{\mathbf{x} - \mathbf{x}_i}{h} \right\|^2 \right)} - \mathbf{x} \quad (3)$$

where  $K' = dK/dx$ , and  $h$  is the bandwidth, which defines the radius of kernel.

The Mean Shift procedure for a given point  $\mathbf{x}$  is as follows [6]:

1. Compute the mean shift vector  $\mathbf{m}(\mathbf{x}^t)$ .
2. Move the density estimation window:  $\mathbf{x}^{t+1} = \mathbf{x}^t + \mathbf{m}(\mathbf{x}^t)$ .
3. Iterate steps 1 and 2 until  $\mathbf{m}(\mathbf{x}^t) < threshold$ .

## 2.3 Bandwidth selection

The selection of bandwidth can heavily influence the performance of the mean shift procedure [8]. If the bandwidth is too small, the kernel window covers only a few points, causing the algorithm to find many local maxima. As a result, the data may be over-clustered into a large number of small clusters. Conversely, if the bandwidth is too large, the window size can smooth out important structure and merge distinct clusters into a single group.

In [2], two automatic and data-based bandwidth selection method are compared. The performance of the selectors is based on the performance of the kernel density gradient estimator, which minimizes a mean squared error. However, the best density estimate does not necessarily give the best clustering, as clustering and density estimation are different problems in natural [2][1]. A practical and computationally simple strategy is to explore clustering behavior over a range of bandwidths.

## 3 Method

The workflow consists of converting circular-linear measurements to Cartesian coordinates, followed by Mean Shift clustering.

### 3.1 Coördiante Transformation

The dataset consists of circular-linear measurements  $(\theta_i, r_i)$ . Since Mean Shift requires Euclidean space, each point is converted to Cartesian coordinates:

$$x_i = r_i \cos(\theta_i), \quad y_i = r_i \sin(\theta_i). \quad (4)$$

## 3.2 Mean Shift in Cartesian Space

Given the transformed data points  $\mathbf{X} = \{(x_i, y_i)\}$ , Mean Shift procedure is iteratively performed as in Section 2.2. Both Epanechnikov and Gaussian kernels are evaluated with multiple bandwidth values to examine clustering stability and performance.

## 4 Results

Clustering results for both kernels across the four bandwidths (0.5, 1.0, 1.5, 2.0) are visualized in Figure 1 and 2 . Each polar scatter plot illustrates the cluster assignments and their corresponding mode locations.

Intuitively, With Epanechnikov kernel, bandwidth 1.5 performs the best, while with Gaussian kernel, bandwidth 0.5 is best.

The statistical properties of each cluster are summarized in Table1 and 2. Both kernels yield 4 clusters and similar clustering patterns. Within each cluster, the covariance matrices have relatively small off-diagonal terms, indicating tight and well-separated clusters.

Table 1: Cluster statistics using Epanechnikov kernel of bandwidth 1.5

| Cluster | #Points | Center (x,y)   | Center (r, $\theta$ ) | Cartesian Covariance Matrix                                  |
|---------|---------|----------------|-----------------------|--------------------------------------------------------------|
| 0       | 941     | (4.81, -0.07)  | (4.81, -0.01)         | $\begin{pmatrix} 0.99 & 0.17 \\ 0.17 & 1.61 \end{pmatrix}$   |
| 1       | 559     | (2.83, 2.74)   | (3.94, 0.77)          | $\begin{pmatrix} 0.65 & -0.16 \\ -0.16 & 0.48 \end{pmatrix}$ |
| 2       | 900     | (-1.69, -0.02) | (1.69, -3.13)         | $\begin{pmatrix} 0.49 & -0.02 \\ -0.02 & 0.28 \end{pmatrix}$ |
| 3       | 500     | (-2.81, 2.72)  | (3.91, 2.37)          | $\begin{pmatrix} 0.47 & 0.19 \\ 0.19 & 0.43 \end{pmatrix}$   |

Table 2: Cluster statistics using gaussian kernel of bandwidth 0.5

| Cluster | #Points | Center (x,y)  | Center (r, $\theta$ ) | Cartesian Covariance Matrix                                  |
|---------|---------|---------------|-----------------------|--------------------------------------------------------------|
| 0       | 950     | (4.74, 0.06)  | (4.74, 0.01)          | $\begin{pmatrix} 0.99 & 0.18 \\ 0.18 & 1.67 \end{pmatrix}$   |
| 1       | 550     | (2.83, 2.76)  | (3.95, 0.77)          | $\begin{pmatrix} 0.59 & -0.17 \\ -0.17 & 0.48 \end{pmatrix}$ |
| 2       | 896     | (-1.76, 0.00) | (1.76, 3.14)          | $\begin{pmatrix} 0.48 & -0.02 \\ -0.02 & 0.28 \end{pmatrix}$ |
| 3       | 504     | (-2.80, 2.73) | (3.91, 2.37)          | $\begin{pmatrix} 0.47 & 0.19 \\ 0.19 & 0.44 \end{pmatrix}$   |

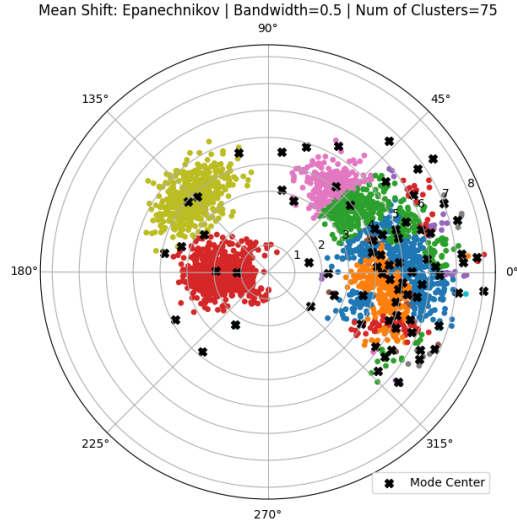
## 5 Conclusion

Mean Shift clustering combined with Cartesian transformation provides an effective approach for analyzing circular-linear data. The method does not require specifying the number of clusters beforehand, making it suitable for exploratory analysis.

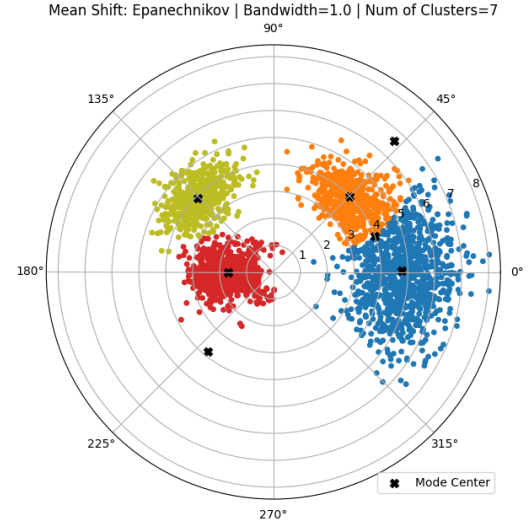
Kernel choice and bandwidth strongly influence the resulting cluster structure; in this study, an Epanechnikov kernel with bandwidth 1.5 and a Gaussian kernel with bandwidth 0.5 produced the clearest cluster structures.

## References

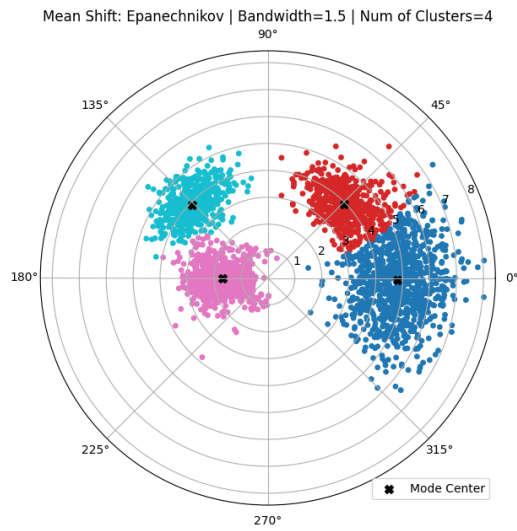
- [1] Miguel A Carreira-Perpinán. A review of mean-shift algorithms for clustering. *arXiv preprint arXiv:1503.00687*, 2015.
- [2] José E Chacón and Pablo Monfort. A comparison of bandwidth selectors for mean shift clustering. *arXiv preprint arXiv:1310.7855*, 2013.
- [3] Yizong Cheng. Mean shift, mode seeking, and clustering. *IEEE transactions on pattern analysis and machine intelligence*, 17(8):790–799, 1995.
- [4] Dorin Comaniciu and Peter Meer. Mean shift analysis and applications. In *Proceedings of the seventh IEEE international conference on computer vision*, volume 2, pages 1197–1203. IEEE, 1999.
- [5] Dorin Comaniciu and Peter Meer. Mean shift: A robust approach toward feature space analysis. *IEEE Transactions on pattern analysis and machine intelligence*, 24(5):603–619, 2002.
- [6] Konstantinos G Derpanis. Mean shift clustering. *Lecture Notes*, 32(1-4):16, 2005.
- [7] Keinosuke Fukunaga and Larry Hostetler. The estimation of the gradient of a density function, with applications in pattern recognition. *IEEE Transactions on information theory*, 21(1):32–40, 1975.
- [8] Kuo-Lung Wu and Miin-Shen Yang. Mean shift-based clustering. *Pattern Recognition*, 40(11):3035–3052, 2007.



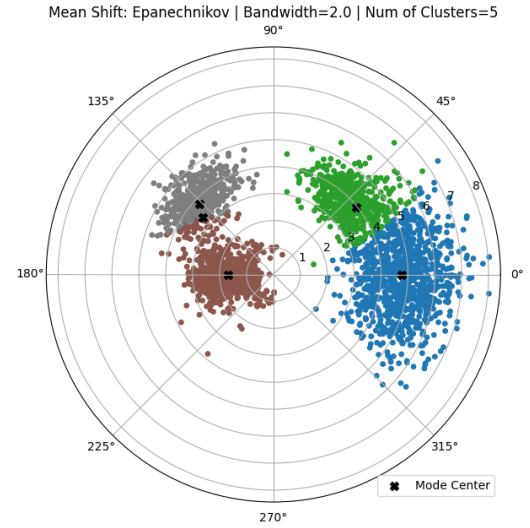
(a) Epanechnikov kernel with bandwidth 0.5



(b) Epanechnikov kernel with bandwidth 1.0

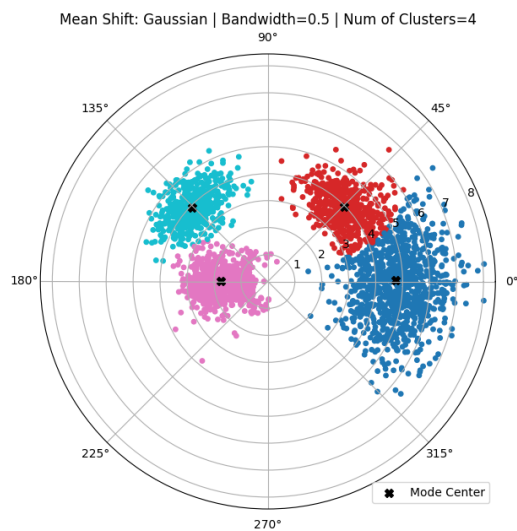


(c) Epanechnikov kernel with bandwidth 1.5

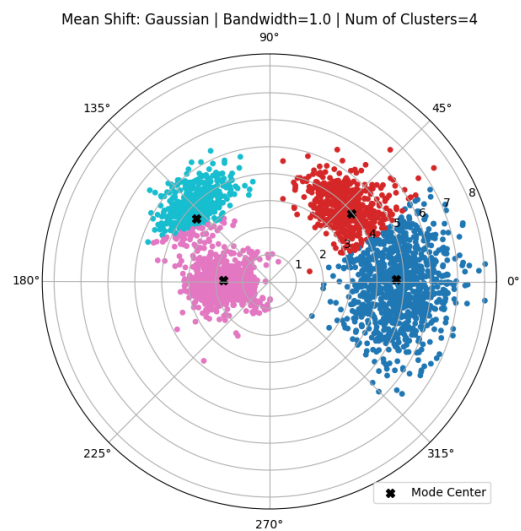


(d) Epanechnikov kernel with bandwidth 2.0

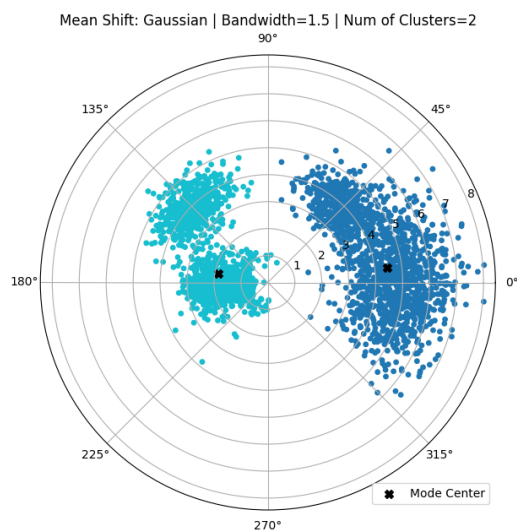
Figure 1: Mean Shift with Epanechnikov kernel with different bandwidths.



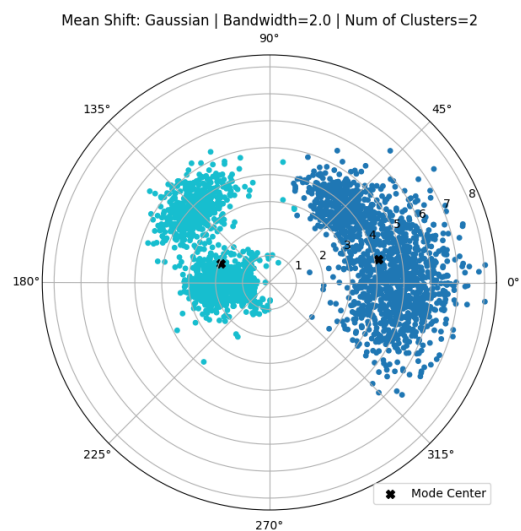
(a) Gaussian kernel with bandwidth 0.5



(b) Gaussian kernel with bandwidth 1.0



(c) Gaussian kernel with bandwidth 1.5



(d) Gaussian kernel with bandwidth 2.0

Figure 2: Mean Shift with Gaussian kernel with different bandwidths.